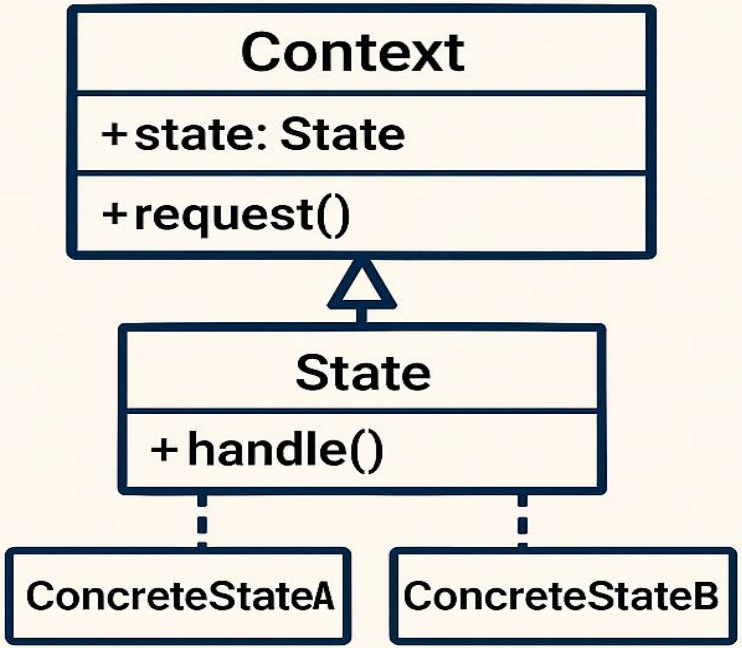
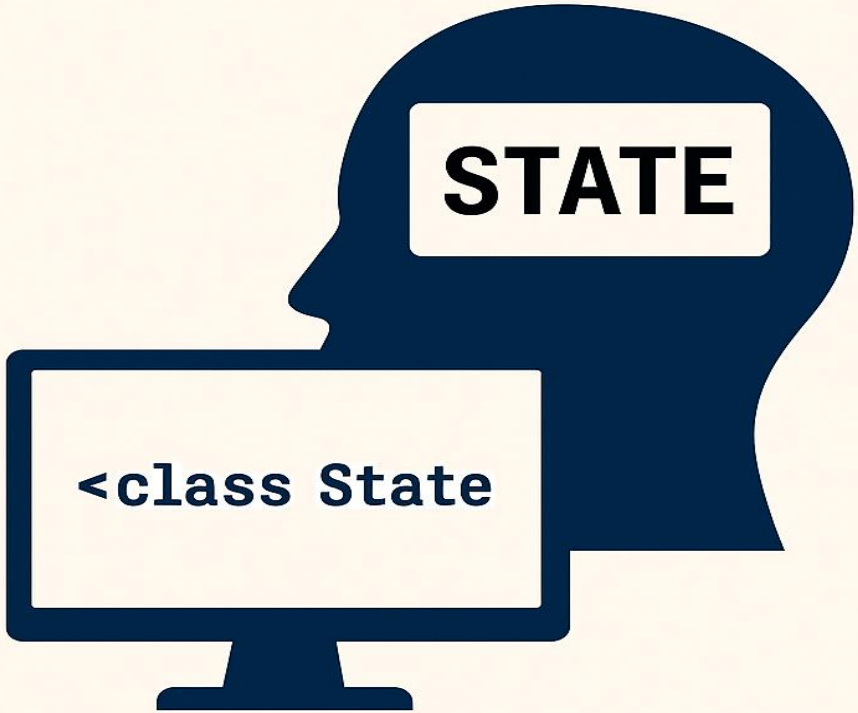
STATE DESIGN PATTERN IN OOP





State Design Pattern

A Behavioral pattern

The state pattern is primarily used when you need to change the behavior of an object based upon the state that it's in at run-time.


AQUARIUS
HOT COFFEE
Enjoy!
Coca-Cola

Creational (5)

- [x] Factory
- [ ] Abstract Factory
- [x] Singleton
- [ ] Prototype
- [ ] Builder

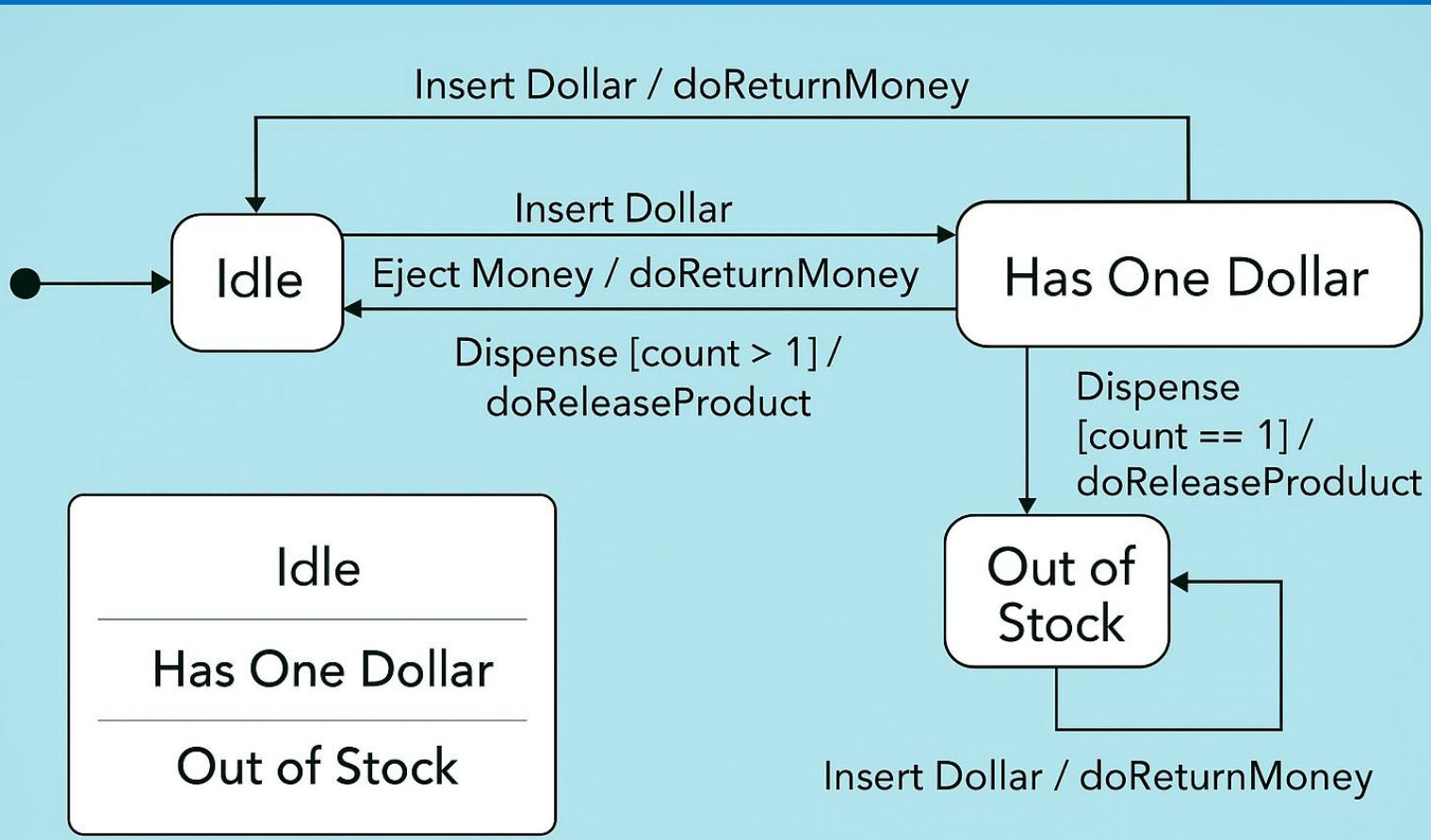
Structural (7)

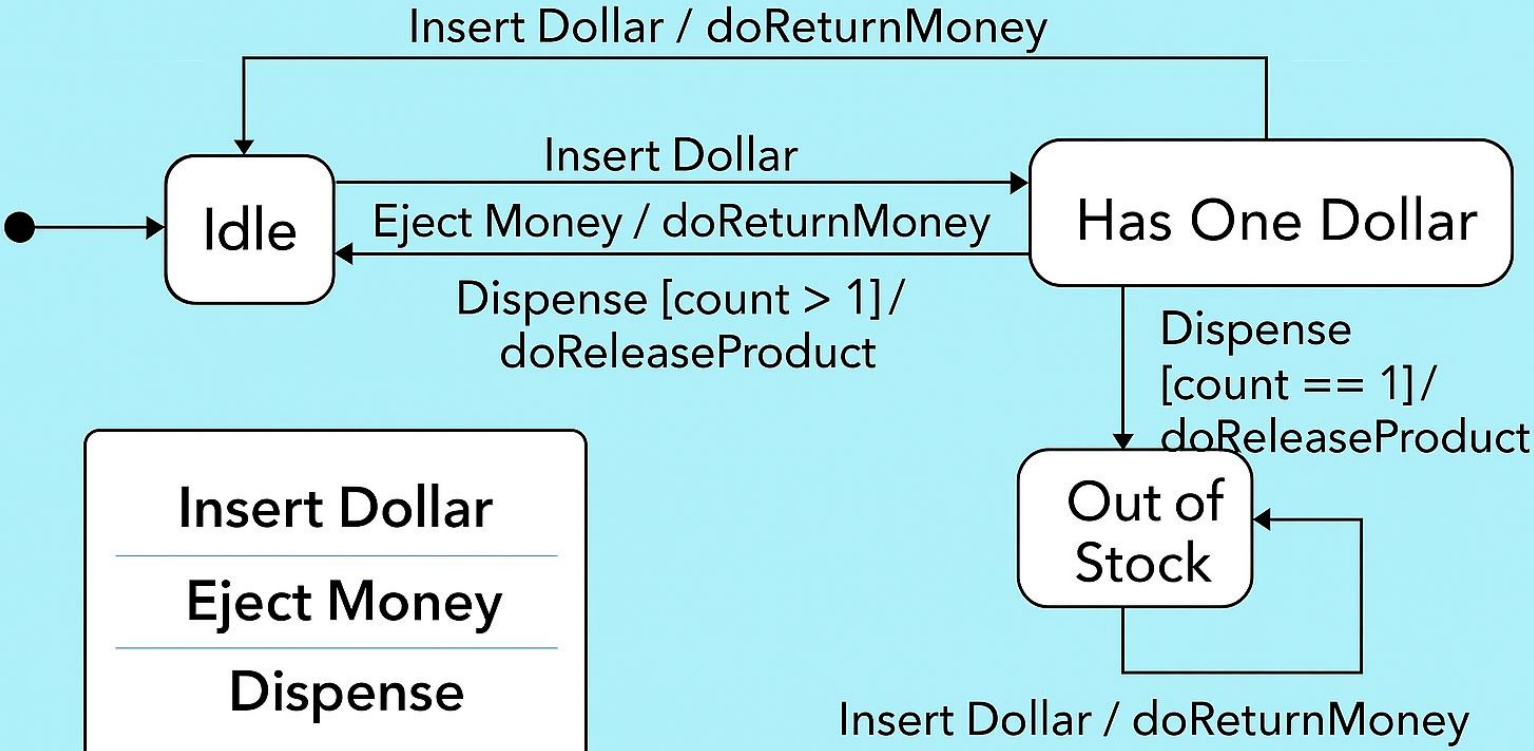
- [x] Adapter
- [ ] Bridge
- [ ] Composite
- [x] Decorator
- [x] Facade
- [ ] Flyweight
- [x] Proxy

Behavioral (11)

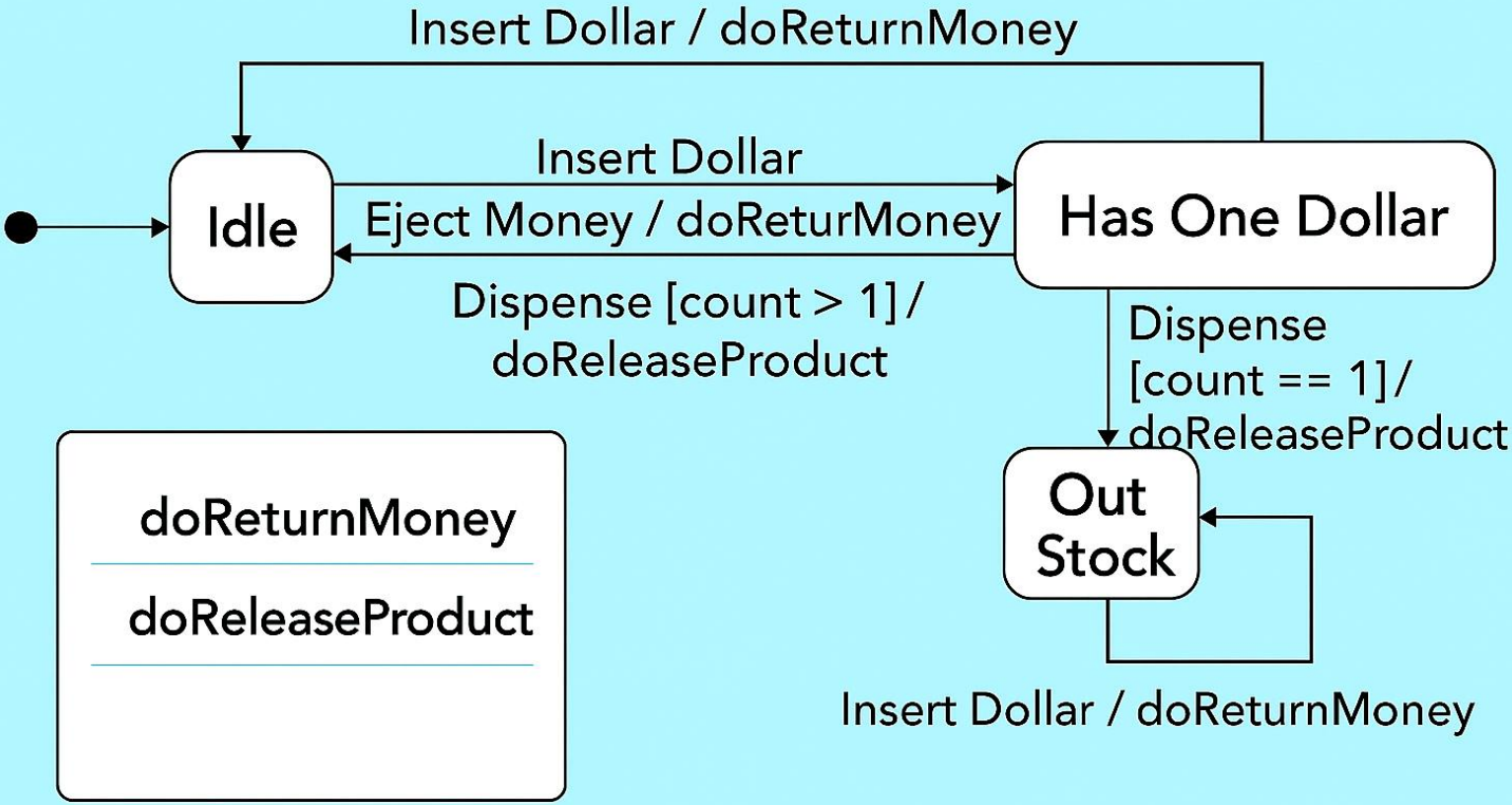
- [x] Chain Of Responsibility
- [ ] Command
- [ ] Interpreter
- [ ] Iterator
- [ ] Mediator
- [ ] Memento
- [ ] Observer
- [ ] State 

- [ ] Strategy
- [x] Template
- [ ] Visitor


Insert Dollar / doReturnMoney
Insert Dollar
Idle
Eject Money / doReturnMoney
Has One Dollar
Dispense [count > 1] /
doReleaseProduct
Dispense
[count == 1] /
doReleaseProduct
Out of
Stock
Insert Dollar / doReturnMoney
Idle
Has One Dollar
Out of Stock



Insert Dollar
Eject Money
Dispense
Insert Dollar
Dispense


Insert Dollar / doReturnMoney
Idle
Insert Dollar
Eject Money / doReturMoney
Has One Dollar
Dispense [count > 1] / doReleaseProduct
Dispense [count == 1] / doReleaseProduct
Out Stock
Insert Dollar / doReturnMoney
doReturnMoney
doReleaseProduct

```
public class VendingMachine {
    …
    public VendingMachine( int count ) {
        …
    }
    // handle user events …
    public void insertDollar() {
        …
    }
    public void ejectMoney() {
        …
    }
    public void dispense() {
        …
    }
    …
}
```

Traditional Implementation

```
final class State { // singleton objects for states
    private State() {}

    // all potential vending machine states as singletons
    public final static State Idle = new State();
    public final static State HasOneDollar = new State();
    public final static State OutOfStock = new State();
}
```

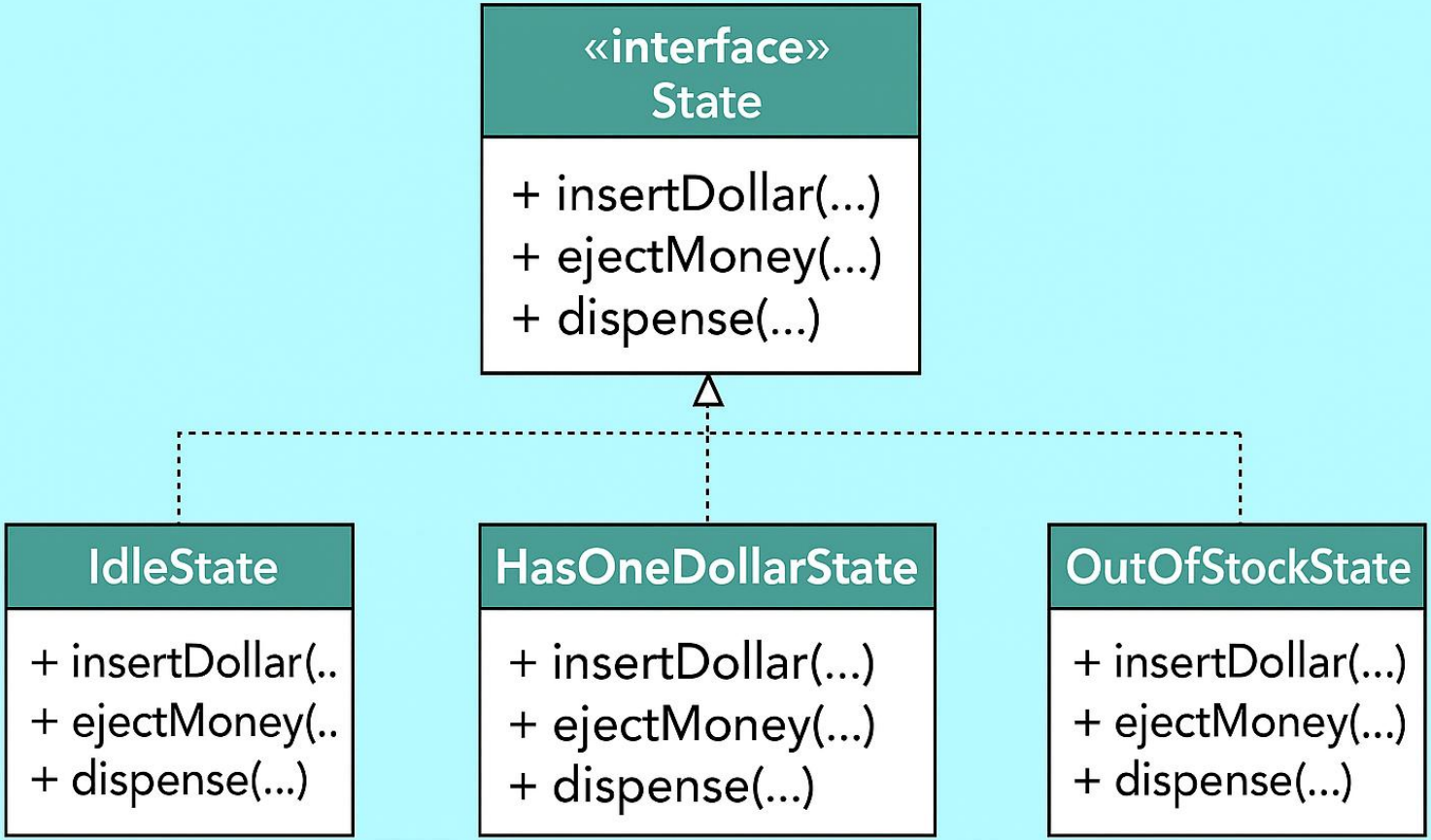
```
public class VendingMachine {

    private State currentState;
    private int count;

    public VendingMachine( int count ) {
        if (count > 0) {
            currentState = State.Idle;
            this.count = count;
        } else {
            currentState = State.OutOfStock;
            this.count = 0;
        }
}
```

```
// handle insert dollar trigger
public void insertDollar() {
     if (currentState == State.Idle) {
          currentState = State.HasOneDollar;
     } else if (currentState == State.HasOneDollar) {
          doReturnMoney();
          currentState = State.Idle;
     } else if (currentState == State.OutOfStock) {
          doReturnMoney();
     }
}
```

State Design Pattern


«interface»
State
+ insertDollar(...)
+ ejectMoney(...)
+ dispense(...)
IdleState
+ insertDollar(..
+ ejectMoney(..
+ dispense(...)
HasOneDollarState
+ insertDollar(...)
+ ejectMoney(...)
+ dispense(...)
OutOfStockState
+ insertDollar(...)
+ ejectMoney(...)
+ dispense(...)

```
public interface State {
    public void insertDollar( VendingMachine vendingMachine );
    public void ejectMoney( VendingMachine vendingMachine );
    public void dispense( VendingMachine vendingMachine );
}
```

```
public class IdleState implements State {

    public void insertDollar( VendingMachine vendingMachine ) {
        System.out.println( "dollar inserted" );

        vendingMachine.setState(
            vendingMachine.getHasOneDollarState()
        );
    }

    public void ejectMoney( VendingMachine vendingMachine ) {
        System.out.println( "no money to return" );
    }

    public void dispense( VendingMachine vendingMachine ) {
        System.out.println( "payment required" );
    }
}
```

```
public class HasOneDollarState implements State {

    public void insertDollar( VendingMachine vendingMachine ) {
        System.out.println( "already have one dollar" );

        vendingMachine.doReturnMoney();
        vendingMachine.setState(
            vendingMachine.getIdleState()
        );
    }

    public void ejectMoney( VendingMachine vendingMachine ) {
        System.out.println( "returning money" );

        vendingMachine.doReturnMoney();
        vendingMachine.setState(
            vendingMachine.getIdleState()
        );
    }
```

```
    public void dispense( VendingMachine vendingMachine ) {
        System.out.println( "releasing product" );

        if (vendingMachine.getCount() > 1) {
            vendingMachine.doReleaseProduct();
            vendingMachine.setState(
                vendingMachine.getIdleState()
            );
        } else {
            vendingMachine.doReleaseProduct();
            vendingMachine.setState(
                vendingMachine.getOutOfStockState()
            );
        }
    }
}
```

```
public class VendingMachine {

    private State idleState;
    private State hasOneDollarState;
    private State outOfStockState;

    private State currentState;
    private int count;

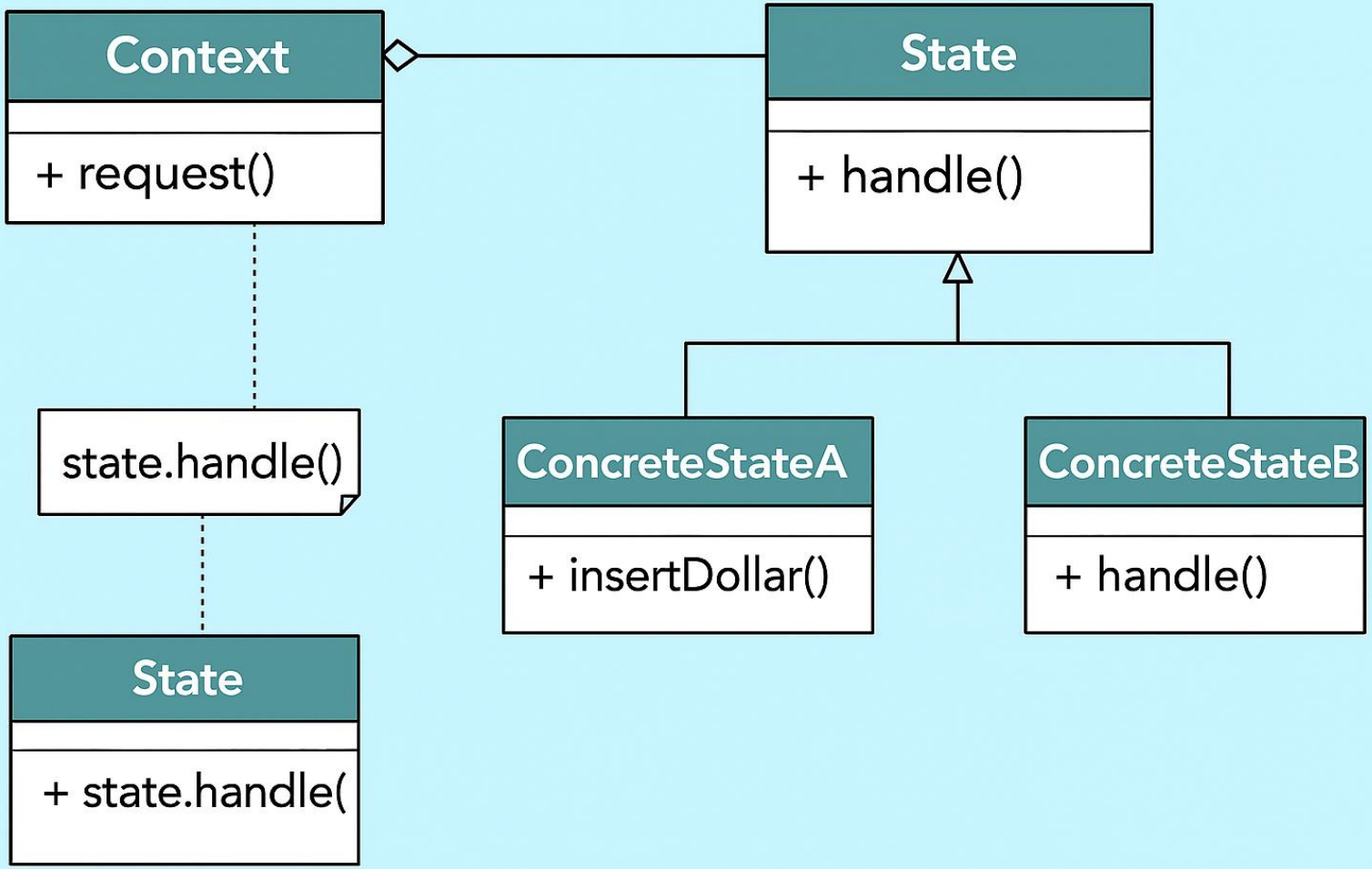
    public VendingMachine( int count ) {
        // make the needed states
        idleState = new IdleState();
        hasOneDollarState = new HasOneDollarState();
        outOfStockState = new OutOfStockState();

        if (count > 0) {
            currentState = idleState;
            this.count = count;
        } else {
            currentState = outOfStockState;
            this.count = 0;
        }
```

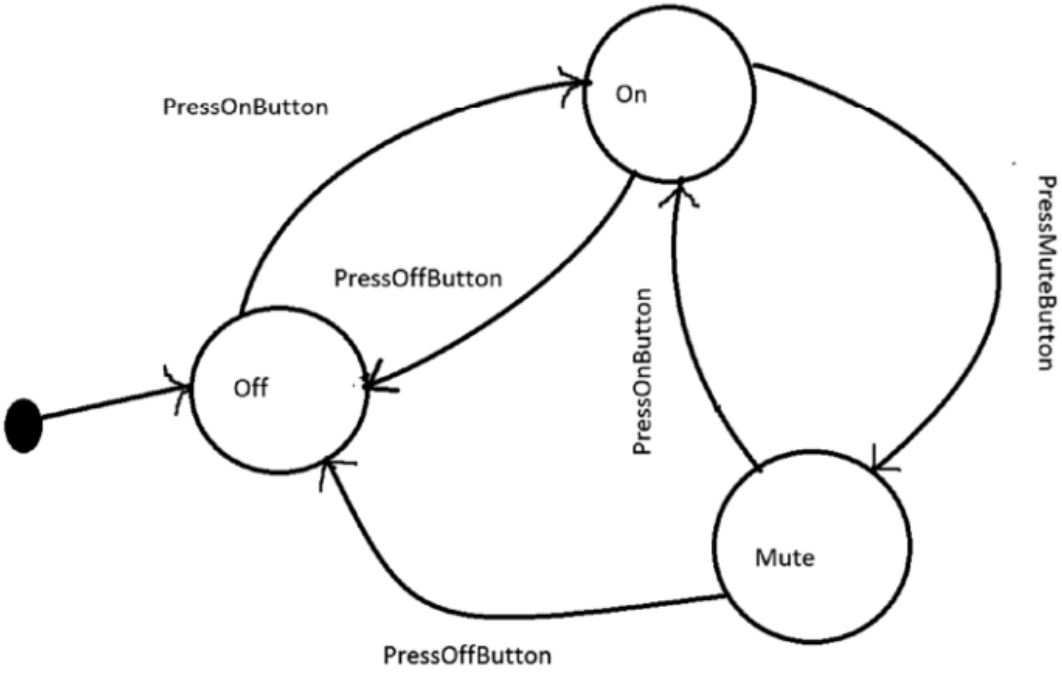
```
public void insertDollar() {
    currentState.insertDollar( this );
}

public void ejectMoney() {
    currentState.ejectMoney( this );
}

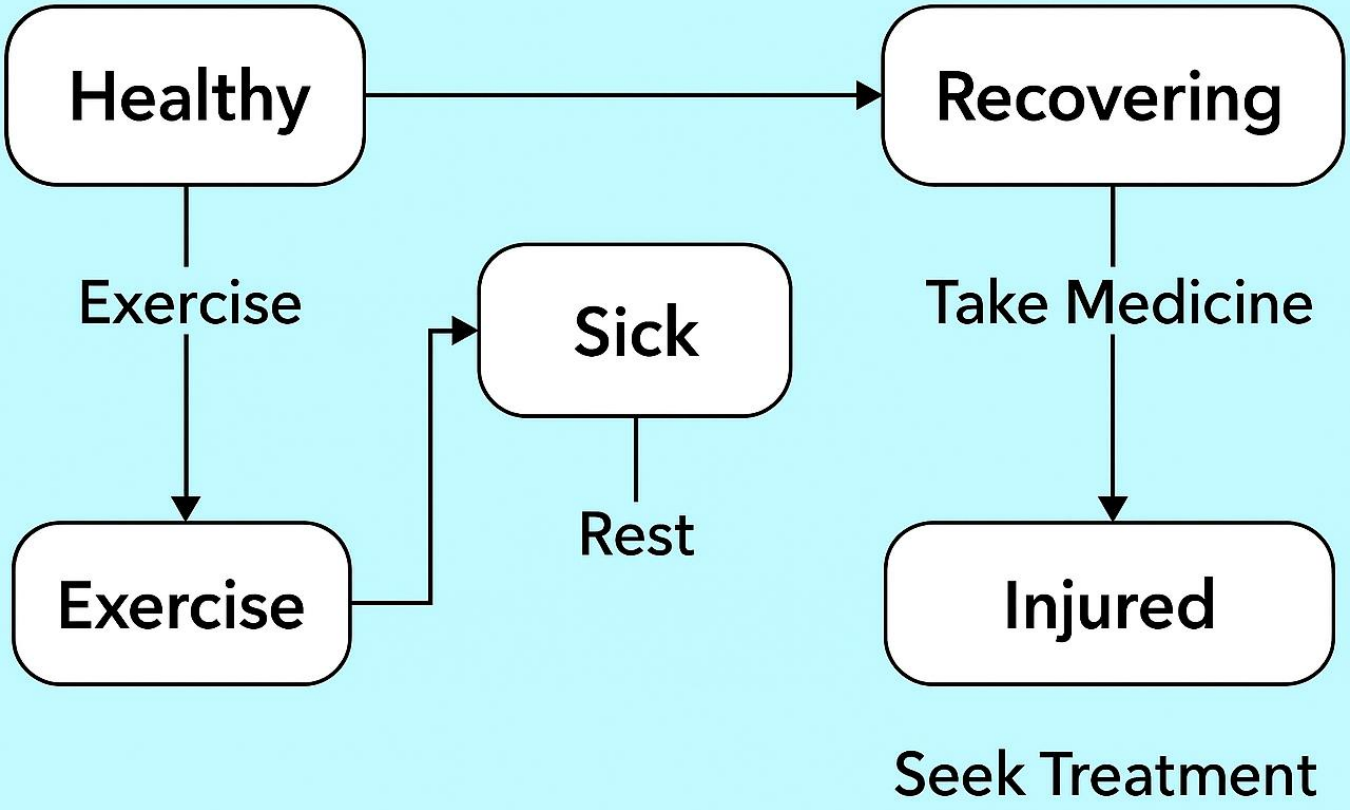
public void dispense() {
    currentState.dispense( this );
}
```

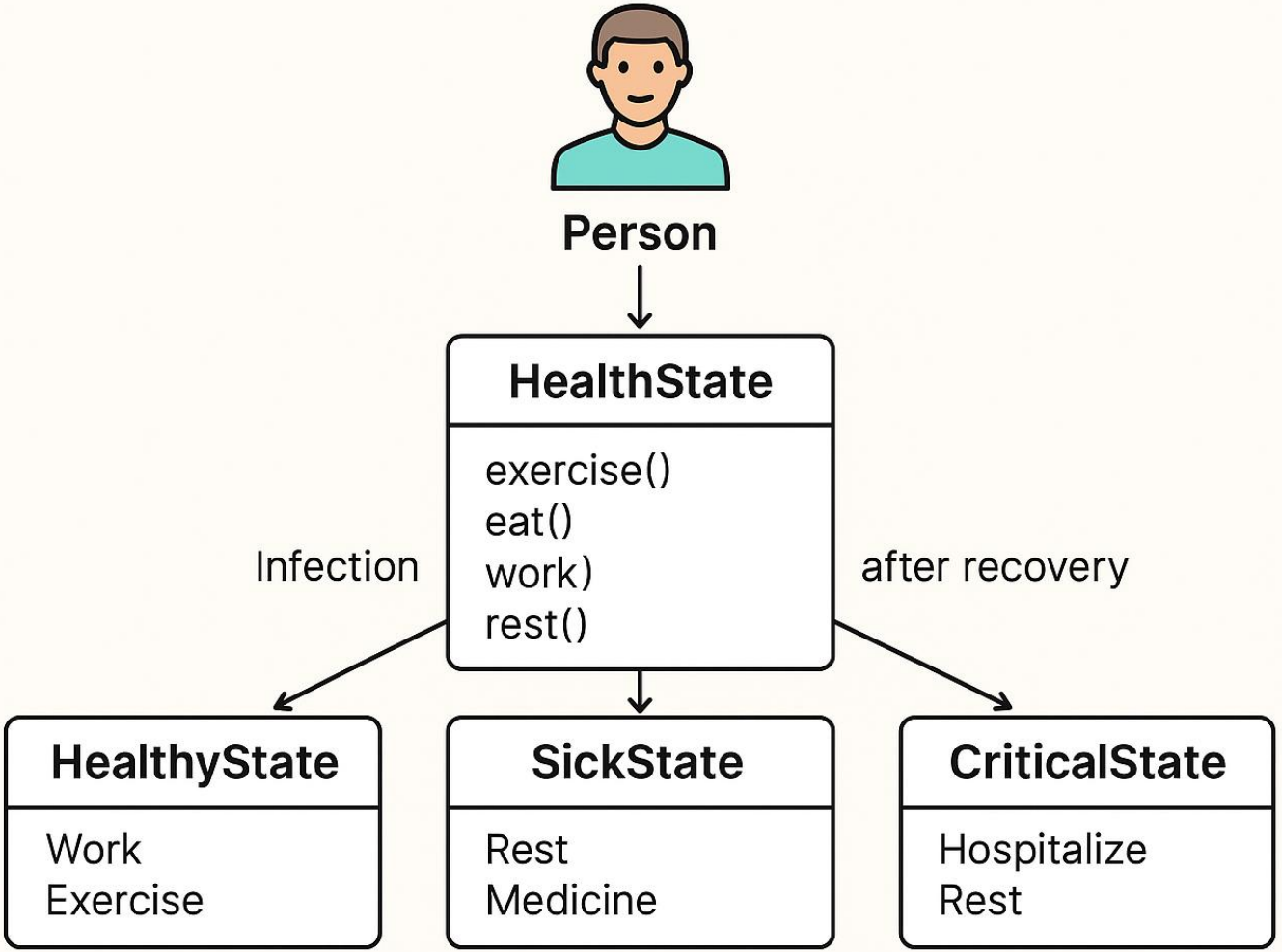

Context
+ request()
State
+ handle()
state.handle()
ConcreteStateA
+ insertDollar()
ConcreteStateB
+ handle()
State
+ state.handle(




On
PressOnButton
PressOffButton
PressOnButton
PressMuteButton
Off
Mute
PressOffButton

State Design Pattern




Person
HealthState
exercise()
eat()
work)
rest()
Infection
after recovery
HealthyState
Work
Exercise
SickState
Rest
Medicine
CriticalState
Hospitalize
Rest

🔑 Key Considerations

Context–State Relationship	•The **Context (e.g., Person)** holds a reference to the current state object. •State transitions should ideally be **encapsulated within state classes**, not exposed directly by the context.
Encapsulation of Behavior	•Each **state class** should define **only the behavior valid in that state**. •Avoid writing conditional logic (if/else or switch) in the context; let polymorphism handle it.
State Transitions	•Ensure **clear rules for transitions** (who controls them? the context or the state?). •Transitions should be **explicit** but not scattered across multiple classes.
Number of States	•Works best when an object has **well-defined finite states** (Healthy, Sick, Critical). •If states are too many or fuzzy, the pattern can overcomplicate.
Extensibility	•Adding a new state should require **minimal changes** to existing code. •This is one of the biggest benefits of the pattern.
Avoiding State Explosion	•If you have too many states and transitions, complexity may rise quickly. •Use grouping or hierarchical states to simplify.
Performance Considerations	•Each action involves **delegation to a state object**. •Usually negligible, but in **performance-critical systems**, you may need to weigh this overhead.

✅ In your health-state example:



The **Person (Context)** delegates behavior (work(), rest(), exercise()) to the current **HealthState**.



Each **state (Healthy, Sick, Critical)** defines what actions are possible.



Transitions (e.g., Sick → Critical, Sick → Healthy) are handled inside state classes, keeping logic clean.

📊 When to Use vs. When Not to Use

✅ When to Use	❌ When Not to Use
An object's **behavior changes significantly** depending on its state (e.g., Person is Healthy, Sick, or Critical).	Object has **very few states** and behavior doesn't change much (simple if conditions are enough).
You want to **eliminate complex if-else or switch statements** that check states repeatedly.	The number of states is **too large**, leading to state-class explosion and complexity.
States are **well-defined, finite, and stable** (don't change frequently).	States are **dynamic, fuzzy, or unbounded**, making state separation impractical.
You want **better organization and readability** by putting state-specific logic in separate classes.	Project is **small/simple** and using multiple classes will add unnecessary overhead.
The system requires **easy extension** (e.g., adding a new state without modifying existing code).	Extending states is **rare**, so the pattern brings more cost than benefit.
State transitions are **part of the domain model** and should be made explicit.	State transitions are **trivial** and don't justify separate state classes.
Useful in **workflow systems, game characters, devices, user accounts, health states** where actions depend on current state.	Not suitable when all behaviors are almost the same regardless of state.

👉 Rule of Thumb

If you find yourself writing **long chains of if (state == …)**, the **State Pattern** is a good fit.

If states are **few and unlikely to change**, stick with simpler logic.

Reflective Questions



How does the State Design Pattern improve code organization compared to using multiple if-else or switch statements?



Can you think of a real-world example (outside the classroom) where the State Pattern could simplify handling different behaviors of an object? Why would this pattern be useful there?



What are the potential drawbacks of using the State Pattern in a system, and how can a developer decide whether it is worth applying in a given situation?